

Insegnare l'ingegneria del software dopo la generazione di codice

Parte I — Dopo la generazione di codice: una riformulazione pratica

Il cambiamento centrale: non insegnare agli studenti a produrre codice. Insegnare loro a formulare problemi, progettare sistemi, valutare evidenze, verificare il comportamento e assumersi la responsabilità del software, che sia stato scritto da una persona, da un'IA o da entrambi. Non si tratta di abbandonare i fondamenti della programmazione, ma di collocarli in una cornice più ampia: l'ingegneria del software come ragionamento sotto vincoli. È una direzione già proposta in letteratura, ad esempio spostando il focus didattico «dalla semplice produzione di codice alla progettazione e all'architettura dei sistemi software» (Azemi, FIE 2025).

1. Il problema è la fiducia immeritata, non l'automazione

L'IA generativa abbassa il costo di produrre codice plausibile. Ciò nonostante, ad oggi, non tutti gli agenti eliminano completamente le parti difficili dell'ingegneria del software: decidere cosa deve fare un sistema, individuare assunzioni implicite, scegliere astrazioni adeguate, capire le dipendenze e i modi in cui un sistema può fallire, testare oltre i casi favorevoli, ragionare su sicurezza, privacy e prestazioni e giustificare le scelte progettuali.

Le linee guida ACM/IEEE-CS/AAAI (CS2023, sezione *Generative AI and the Curriculum*) sono esplicite: anche se l'IA generativa può scrivere programmi semplici, «la responsabilità di verificare la correttezza dei programmi ricade ancora sull'utente»; per questo, anche solo ai fini della verifica, gli studenti di informatica devono ancora imparare a scrivere programmi (ACM/IEEE-CS/AAAI CS2023 Task Force, bozza gen. 2024; versione apr. 2024, §4.3). Lo stesso documento prevede che l'apprendimento si sposti «dallo scrivere» verso il «promptare, comprendere, verificare, modificare, adattare e testare» codice. Una tesi di dottorato recente arriva a conclusioni simili dal lato professionale: gli sviluppatori intervistati considerano l'IA generativa utile soprattutto per la scrittura del codice, ma ritengono insostituibili le conoscenze fondamentali di ingegneria del software (Wang, Virginia Tech, 2026).

Questo crea una pericolosa asimmetria: capacità di generare codice » capacità di comprenderlo. Uno studente può consegnare un artefatto funzionante senza possederne il modello mentale corrispondente. È pericoloso, perché «funziona sull'esempio» è una definizione debole di correttezza.

Principio: ogni artefatto generato dovrebbe essere accompagnato da un'argomentazione sul perché ci si può fidare: test, invarianti, modelli di minaccia, analisi di complessità, alternative considerate oppure una revisione documentata.

2. L'intuizione di Knuth conta più di prima

Knuth proponeva di cambiare atteggiamento verso la costruzione dei programmi: «invece di immaginare che il nostro compito principale sia istruire un computer su cosa fare, concentriamoci piuttosto sullo spiegare a esseri umani cosa vogliamo che un computer faccia». Chiamava questo approccio *literate programming*, programmazione letteraria (Knuth, *Literate Programming*, The Computer Journal 27(2), 1984, p. 97). L'IA rende questa idea più importante, non meno. Quando digitare la sintassi costa poco, la competenza scarsa diventa produrre e valutare spiegazioni. Perché questa architettura? Quali assunzioni fa questa funzione? Quali proprietà devono valere sempre? Quali prove sostengono la correttezza? Cosa potrebbe sfruttare un attaccante? Cosa succede se i requisiti cambiano?

L'unità di insegnamento dovrebbe allargarsi dalla funzione all'argomentazione software. Invece di «implementa questo algoritmo su grafo», si può chiedere:

1. una specifica;
2. un'implementazione, eventualmente assistita dall'IA;
3. affermazioni di correttezza;
4. test avversariali o basati su proprietà;
5. un'analisi di complessità;
6. un'analisi dei modi di fallimento;
7. un confronto con un design alternativo;
8. un breve resoconto di cosa è stato generato e cosa scritto a mano.

Il codice resta importante, ma non è più l'intero oggetto di valutazione. Coerentemente, CS2023 si aspetta che gli studenti sappiano valutare i risultati dell'IA generativa dal punto di vista dei compromessi tempo-spazio e dell'analisi di complessità (CS2023, §4.3).

3. Le evidenze sono contrastanti, ed è utile che lo siano

- **Produttività su compiti ristretti.** In un esperimento controllato condotto da ricercatori di GitHub, Microsoft Research e MIT, 95 sviluppatori freelance sono stati assegnati casualmente a due gruppi e hanno implementato un server HTTP in JavaScript. Il gruppo con GitHub Copilot ha completato il compito il 55,8% più velocemente del gruppo di controllo (IC 95%: 21–89%). Si tratta però di un compito singolo e vincolato, non di un indicatore di competenza a lungo termine (Peng et al., 2023; riassunto).
- **Produttività su repository reali.** In uno studio randomizzato di METR, 16 sviluppatori open source esperti hanno svolto 246 task su repository maturi che conoscevano bene. Con strumenti di IA (inizio 2025) hanno impiegato il 19% di tempo in più. Prima dello studio prevedevano un'accelerazione del 24%; a posteriori stimavano che l'IA li avesse resi più veloci del 20% (Becker et al., METR, 2025; resoconto divulgativo). *Aggiornamento:* nel febbraio 2026 METR ha riportato che un nuovo esperimento, condotto a fine 2025, è pesantemente distorto da selezione: il 30–50% degli sviluppatori evitava di consegnare task da svolgere senza IA. Per questo METR considera quei dati un segnale «inaffidabile» e sta riprogettando lo studio (METR, 24/02/2026†). Il risultato del 2025 va quindi letto come una fotografia di quel momento, non come una legge generale.
- **Apprendimento.** Il dato più rilevante per la didattica viene da uno studio controllato su 24 studenti universitari, principianti e intermedi, che ha confrontato l'uso di ChatGPT con le normali risorse online. Generare soluzioni complete con l'IA generativa migliora significativamente la prestazione nel compito, soprattutto per i principianti, ma non produce in modo consistente guadagni di conoscenza. I principianti tendono ad appoggiarsi molto all'IA per completare il compito, spesso senza imparare. Sia l'uso eccessivo sia l'uso minimo portano a guadagni di conoscenza più deboli (Chen et al., 2025).
- **Quadro più ampio.** Due revisioni sistematiche, una su 60 e una su 45 studi empirici, trovano benefici (supporto al coding e al debug, coinvolgimento, fiducia) insieme a rischi ricorrenti di apprendimento superficiale e dipendenza. Concludono che l'esito dipende da come l'IA viene integrata nella didattica (Yalwa et al., IJACSA 2026; Huang & Tseng, IJIEET 2026). In un'indagine su 284 studenti di informatica non è emersa alcuna correlazione significativa tra uso dell'IA e competenze di programmazione percepite (Beralde et al., IJRSI 2026). Altri studenti ammettono di affidarsi a ChatGPT proprio per i compiti che richiedono pensiero analitico (Melgarejo-Solis et al., LACCEI 2025). Sul versante positivo, in corsi introduttivi che combinano

flipped learning e IA generativa si sono osservati miglioramenti nel pensiero computazionale e nel problem solving (Jang & Oh, 2026). Con un protocollo di uso guidato, poi, gli studenti valutano criticamente, modificano e rifiutano più spesso i suggerimenti dell'IA a parità di prestazione (Grewé et al., CSEDU 2026).

Principio progettuale: uno studente può completare più esercizi acquisendo una comprensione meno trasferibile. Al contrario, il tempo speso a fare debug o a confrontare alternative può avere valore didattico anche quando riduce la produttività immediata.

4. Idea: come valutare i progetti?

Due studenti possono consegnare codice indistinguibile con livelli di comprensione radicalmente diversi. Una breve discussione orale (non un interrogatorio) può chiedere allo studente di:

- modificare un requisito e prevederne le conseguenze;
- spiegare una funzione non ovvia;
- indicare il test più debole;
- dimostrare un fallimento;
- difendere una scelta progettuale;
- spiegare un suggerimento dell'IA che ha rifiutato.

Lo scopo è verificare se lo studente padroneggia davvero il sistema o si limita a possederlo.

Anche le evidenze empiriche puntano in questa direzione. In un'indagine su 130 studenti di ingegneria del software, il 79% è favorevole a riprogettare le consegne tenendo conto dell'IA. Gli autori raccomandano esami orali, revisioni del codice o saggi riflessivi in cui gli studenti spiegano *come* hanno usato l'IA, non solo cosa ha prodotto (Qin et al., 2025). In progetti *capstone* con clienti reali, 7 clienti su 11 hanno segnalato come problema la scarsa comprensione da parte degli studenti (Mircea et al., 2026). Altri strumenti complementari sono le dichiarazioni d'uso dell'IA, i compiti di validazione dell'output e la revisione tra pari degli artefatti generati (Garousi et al., JSS 2026).

5. Insegnare i requisiti prima dei prompt

Il *prompt engineering* è secondario. Un prompt vago di solito riflette una specifica assente, non una formulazione infelice. Le linee guida CS2023 osservano che, per dare all'IA indicazioni appropriate, gli studenti devono saper progettare e pianificare programmi di una certa dimensione, e prevedono una maggiore attenzione alla scomposizione dei problemi (CS2023, §4.3). Gli studenti dovrebbero imparare a esprimere:

- attori e obiettivi;
- precondizioni e postcondizioni;
- invarianti;
- requisiti non funzionali;
- condizioni di errore;
- vincoli di risorse;
- confini di fiducia;
- requisiti di privacy;
- criteri di accettazione.

Solo dopo dovrebbero chiedere all'IA suggerimenti implementativi. Anche gli studenti stessi, pur usando l'IA molto spesso, riconoscono di comprendere poco le tecniche di prompting che adottano, e ne indicano l'inaccuratezza come principale ostacolo (Hidalgo García et al., CECIS 2025).

Esempio. Invece di «scrivi un servizio di autenticazione sicuro», si richiede una specifica in cui:

- le password non vengono mai salvate in chiaro;
- i tentativi di accesso sono soggetti a *rate limiting*;
- i token di reset sono monouso e scadono;
- i log non contengono mai credenziali;
- il servizio resiste ad attacchi di *replay*;
- la latenza resta sotto una soglia data;
- il modello di minaccia copre *credential stuffing* e furto di token.

Il prompt diventa un'interfaccia verso un design che lo studente già comprende.

6. Il principio più profondo (finora)

Il curriculum CS2023 definisce la competenza come conoscenze + abilità + disposizioni. Articola le abilità nei livelli *spiegare*, *applicare*, *valutare*, *sviluppare* e intende le disposizioni come valori e atteggiamenti professionali: adattabilità, collaborazione, meticolosità, perseveranza ecc. Non si limita quindi alla produzione di output (Kumar et al., *Computer Science Curricula 2023*, ACM 2024; *presentazione CS2023*; IEEE Computer Society, *comunicato 2024*). Questa cornice conta ora più che mai: l'IA può produrre artefatti, ma l'educazione deve costruire la disposizione a metterli in discussione. Studi empirici su curricula e docenti vanno nella stessa direzione. L'uso dell'IA non si può impedire in modo affidabile, e non sarebbe nemmeno desiderabile farlo, quindi i curricula dovrebbero integrarla, anche negli esami (Randall et al., *IEEE Software* 41(2), 2024). Stanno già nascendo corsi dedicati allo sviluppo assistito dall'IA. In un'analisi di 23 syllabi, i temi più frequenti tra gli obiettivi di apprendimento sono la collaborazione uomo-IA, lo sviluppo di strumenti e la *valutazione* di software e artefatti IA (correttezza, manutenibilità, affidabilità). Temi come l'uso responsabile compaiono invece più di rado (Geng et al., 2026). Resta invece poco studiato l'uso dell'IA nei corsi avanzati, oltre quelli introduttivi (Bouvier et al., ITiCSE-WGR 2025).

- **Uso debole:** «Costruisci questa applicazione.»
- **Uso più forte:** «Proponi tre architetture per questo sistema specificato. Indica le assunzioni dietro ciascuna, identifica i probabili modi di fallimento, genera test che possano distinguerle, e spiega quale raccomandazione rifiuteresti e perché.»

Il secondo uso non externalizza il giudizio: trasforma l'IA in un oggetto di critica e confronto. Gli studenti dovrebbero uscire sapendo non solo come chiedere codice all'IA, ma anche come chiedersi se quel codice merita di esistere. Le evidenze finora suggeriscono che l'IA può aumentare il completamento dei compiti senza produrre guadagni di conoscenza affidabili, soprattutto per i principianti (Chen et al., 2025). L'obiettivo, quindi, non è né la restrizione massima né l'automazione massima. È coltivare comprensione autonoma, fiducia calibrata, verifica rigorosa, giudizio progettuale e collaborazione responsabile con le macchine.

La Parte I, però, inquadra ancora l'obiettivo come «insegnare ciò che l'IA non sa (ancora) fare». La Parte II va oltre questa cornice e si chiede cosa succede quando questa premessa smette di valere.

Parte II — E se l'IA facesse meglio? Sulla giustificazione più profonda dell'educazione all'ingegneria del software

Questa parte parte da due obiezioni dirette all'intera premessa della Parte I:

1. Tutto ciò che oggi si dice che l'IA non sappia fare: supponiamo che domani lo sappia fare, e meglio di noi. Cosa resta dell'argomento per insegnarlo? Non è un'ipotesi di scuola. Tra i ricercatori più noti c'è chi la ritiene probabile: Geoffrey Hinton, ad esempio, ha dichiarato di sospettare che l'IA «alla fine diventerà migliore di noi, in tutto», anche se «una cosa alla volta» ([Hinton a StarTalk, 28/02/2026, trascrizione](#)).
2. Oggi quasi nessuno calcola più le radici quadrate a mano, e nessuno se ne addolora. Domani le persone potrebbero non avere più bisogno dell'ingegneria del software: perché trattare la sua scomparsa come una crisi?

«Insegnare agli studenti a fare ciò che l'IA non sa fare» non è un principio stabile, perché il confine delle capacità dell'IA si sposta continuamente. Se l'IA finisse per superare gli esseri umani nell'analisi dei requisiti, nell'architettura, nei test, nella revisione di sicurezza e persino nella scoperta scientifica, quelle attività non si potrebbero difendere solo perché oggi sono punti di forza umani. La domanda più forte non è se gli umani debbano restare migliori dell'IA nell'ingegneria del software. È che tipo di esseri, cittadini e istituzioni vogliamo che gli umani diventino quando l'ingegneria del software potrà essere in gran parte automatizzata. È la stessa domanda che la filosofia dell'educazione si sta ponendo più in generale: cosa significa coltivare menti quando i compiti cognitivi si possono delegare alle macchine ([Bialystok, Educational Theory, 2025; copia su Scribd](#)).

1. L'analogia della radice quadrata è corretta, e ha un limite

Nessuno pretende che una persona istruita sappia calcolare le radici quadrate a mano: usiamo calcolatrici e librerie. La competenza rilevante non è l'esecuzione manuale. È saper riconoscere quale problema si sta risolvendo, stimare se una risposta è plausibile, scegliere il metodo appropriato, comprendere i limiti di uno strumento e accorgersi di un uso improprio. Lo stesso potrebbe valere per la programmazione: gli studenti potrebbero non aver bisogno di implementare a mano ogni struttura dati, parser o endpoint CRUD.

Ma l'analogia ha un limite. Una radice quadrata è un'operazione matematica ben definita; i sistemi software sono costruzioni socio-tecniche con obiettivi ambigui, interessi contrastanti, rischi di sicurezza, istituzioni e conseguenze irreversibili. Una calcolatrice può sbagliare per un errore numerico. Un sistema medico, finanziario o di sicurezza generato dall'IA può sbagliare in modi che cambiano la vita delle persone. Il problema più profondo, quindi, non è la produzione manuale ma il giudizio situato e la responsabilità.

2. Cosa resta se l'IA diventa migliore in tutto?

Supponiamo che l'IA finisca per superare gli umani in programmazione, architettura, test, debug, analisi di sicurezza, raccolta dei requisiti, gestione dei progetti e modellazione scientifica. L'«ingegneria del software umana» potrebbe scomparire come necessità occupazionale. Anche in quel caso restano diverse cose, e nessuna è una competenza cognitiva esclusivamente umana:

- **Capacità d'agire (agency).** Chi decide cosa costruire? Un'IA può ottimizzare un obiettivo, ma gli obiettivi non si scoprono nel vuoto. «Cosa dovrebbe ottimizzare questo sistema?» è una domanda politica ed etica prima che tecnica.
- **Responsabilità (accountability).** Chi risponde quando il sistema danneggia qualcuno? Se un sistema progettato dall'IA nega l'accesso alle cure, perde dati o classifica erroneamente una persona come minaccia,

«l'ha generato il modello» non è una risposta adeguata. La responsabilità può essere distribuita tra aziende, operatori, regolatori e utenti, ma non scompare solo perché l'implementazione è automatizzata.

- **Legittimazione.** Anche una soluzione superiore ha bisogno dell'accettazione umana. Una comunità può rifiutare un algoritmo non perché sia tecnicamente inferiore, ma perché viola autonomia, dignità, privacy, equità o controllo democratico. Ottimale dal punto di vista tecnico non significa automaticamente legittimo.
- **Significato e impegno.** Una macchina può produrre un design, ma questo non significa che quel design abbia significato per le persone che dovranno convivervi. L'educazione riguarda in parte la formazione di impegni: cosa vale la pena proteggere, quali rischi ci si rifiuta di imporre agli altri, quali istituzioni si è disposti a costruire.
- **Distribuzione del potere.** Chi possiede i modelli, i dati, la potenza di calcolo e le infrastrutture? Chi beneficia dell'automazione, e chi ne diventa dipendente? Forse è la domanda più profonda: non solo «l'IA può fare ingegneria del software?», ma chi decide cosa fa l'intelligenza automatizzata, nell'interesse di chi e con quale possibilità di rifiuto. Un'automazione totale del lavoro software potrebbe essere liberatoria, oppure concentrare un potere senza precedenti in poche aziende e stati. L'antologia UNESCO 2025 sul futuro dell'educazione mette al centro proprio questi temi: accesso diseguale ai modelli più avanzati, quali valori e conoscenze i sistemi incorporano, equilibrio tra autonomia e dipendenza (UNESCO, *AI and the future of education*, 2025). Anche sul piano economico la scelta tra un'IA che sostituisce il lavoro e una che ne estende il giudizio non è neutra: secondo Acemoglu, Autor e Johnson, fallimenti di mercato e *bias* ideologici spingono verso l'automazione (Acemoglu, Autor & Johnson, NBER w34854, 2026).

3. I migliori pensatori non convergono su un'unica risposta

Nessuna singola tradizione risolve la questione, ma diverse ne illuminano aspetti diversi. (Nota: nel documento originale questa sezione non aveva fonti. Le fonti † sono state aggiunte in fase di verifica.)

- **Platone/Socrate: l'educazione non è trasmissione di informazioni.** Nel *Menone* il dialogo si chiede se la virtù possa essere insegnata. Nel celebre episodio dello schiavo, Socrate lo porta a scoprire una proprietà geometrica (come raddoppiare l'area di un quadrato) ponendogli domande invece di fornirgli la risposta. La perplessità è presentata come una tappa salutare verso la conoscenza (SEP, *Plato's Ethics*†). Nel *Fedro* Socrate diffida della scrittura: essa produrrebbe «oblio» in chi la usa e darebbe «l'apparenza della sapienza, non la sapienza vera», perché le parole scritte, una volta maltrattate, «hanno sempre bisogno dell'aiuto del padre» e non sanno difendersi da sole (*Fedro* 275a–e, trad. Fowler†). La versione moderna è questa: uno studente può ottenere dall'IA una spiegazione corretta senza acquisire la capacità di ragionare in modo autonomo. Il problema non è che gli strumenti siano illegittimi, ma che avere accesso alle risposte non equivale a comprendere. L'educazione dovrebbe coltivare l'esame delle ragioni e la partecipazione all'indagine, non solo l'accumulo di output.
- **Aristotele: l'educazione forma il giudizio pratico.** Aristotele distingue il *fare-produrre* (*poiesis*) dall'*agire* (*praxis*). La *techne* (arte, tecnica) è una disposizione razionale rivolta alla produzione; la *phronesis* (saggezza pratica) riguarda il deliberare bene su ciò che è buono per la vita nel suo complesso (*Etica Nicomachea* VI, 1140a–b, trad. Rackham†). Nessun insieme di regole, per quanto complesso, risolve ogni problema pratico (SEP, *Aristotle's Ethics*†). Un'IA può avere una competenza tecnica straordinaria, ma qualcuno deve comunque decidere quale compromesso sia accettabile, quali interessi contino, quale rischio sia tollerabile, se un requisito sia di per sé ingiusto e quando non automatizzare. Sono decisioni su come vivere insieme, non solo su come produrre un artefatto.
- **Heidegger: il pericolo è ridurre tutto a risorsa.** Per Heidegger la proliferazione dei dispositivi tecnologici è un fenomeno superficiale, che nasconde un mutamento più profondo nel modo in cui le cose ci si

presentano. Nell'epoca della tecnica gli enti, *compresi gli esseri umani*, appaiono come *Bestand*, «fondo» o «scorta»: pezzi disponibili, sostituibili e ordinabili all'interno del *Gestell* (l'«impianto»). La fonte è *Die Frage nach der Technik*, conferenza del 1953 pubblicata nel 1954 (SEP, [Martin Heidegger](#), §5.2†). In un ambiente di lavoro saturo di IA le persone rischiano di diventare prompt da elaborare, lavoratori da misurare, utenti da prevedere, studenti da valutare. Le istituzioni potrebbero attribuire sempre più valore a ciò che può essere generato e misurato, trascurando ciò che resiste a queste categorie.

- **Hannah Arendt: pensare non è la stessa cosa che essere intelligenti.** Arendt distingue la capacità intellettuale dall'attività del pensare, cioè il «dialogo silenzioso» con sé stessi con cui esaminiamo ciò che facciamo (SEP, [Hannah Arendt](#)†). Nelle sue parole: «L'incapacità di pensare non è stupidità; la si può trovare in persone estremamente intelligenti» (Arendt, *Thinking and Moral Considerations*, *Social Research* 38(3), 1971†). Un sistema può essere più intelligente di una persona in senso strumentale, mentre la persona resta responsabile di chiedersi se le proprie azioni valgano la pena. La preoccupazione di Arendt per la mancanza di pensiero (*thoughtlessness*) si applica al lavoro assistito dall'IA: le persone possono diventare molto efficienti in processi di cui non esaminano mai il significato.
- **Ivan Illich: gli strumenti possono potenziare o disabilitare.** In *Tools for Conviviality* (1973) Illich distingue gli strumenti «conviviali», che ampliano l'autonomia e la creatività di chi li usa, dagli strumenti industriali. Questi ultimi, oltre una certa soglia, finiscono per usare le persone invece di essere usati da loro e creano dipendenza da istituzioni ed esperti (*Tools for Conviviality*, [Wikipedia](#)†). L'analogia della radice quadrata mostra il lato che potenzia. Ma se gli studenti perdono ogni capacità di stimare o di procedere senza calcolatrice, la dipendenza prende il posto del potenziamento. Un tutor IA può ampliare l'accesso alla spiegazione, e un assistente di programmazione può aiutare un principiante a esplorare idee. Gli stessi strumenti possono però lasciare le persone incapaci di agire, capire o riparare qualcosa senza la piattaforma. La vera domanda non è «strumento sì o no», ma se lo strumento amplia la capacità d'agire o la rende opaca e fragile.
- **Bernard Stiegler: gli strumenti costituiscono le capacità umane.** Per Stiegler (*La technique et le temps*) la tecnica non è esterna all'umano ma lo costituisce. Memoria e intelligenza sono sempre state sostenute da supporti tecnici esterni («ritenzioni terziarie»: scrittura, registrazioni, computer), che non si limitano ad assistere capacità preesistenti ma ridisegnano ciò che gli umani possono diventare. Ogni tecnica è inoltre un *pharmakon*, al tempo stesso veleno e rimedio ([Bernard Stiegler](#), [Wikipedia](#)†). È una cornice più ottimistica: non esiste un'essenza umana incontaminata minacciata dall'IA. Ma ogni supporto tecnico cambia anche abitudini e istituzioni. La sfida educativa è fare in modo che l'IA diventi un mezzo per sviluppare capacità, non un sostituto del possederle.

4. Perché preoccuparsi se l'ingegneria del software scomparisse? Quattro ragioni

Nessuna di queste equivale a «perché gli umani devono continuare a fare per sempre gli stessi lavori»:

1. **Le generazioni di transizione hanno bisogno di competenza.** Anche se l'automazione totale fosse un giorno possibile, gli studenti attraverseranno un lungo periodo di automazione parziale, sistemi inaffidabili e dipendenza istituzionale. Devono capire abbastanza da supervisionare, mettere in discussione, riparare e governare i sistemi durante quella transizione.
2. **La capacità influenza il potere.** Il software è infrastruttura: governa comunicazione, finanza, sanità, energia, trasporti, istruzione e amministrazione pubblica. Se solo una piccola élite tecnica, o pochi fornitori di IA, comprende come funzionano questi sistemi, tutti gli altri ne diventano politicamente dipendenti. L'alfabetizzazione software di base è più vicina all'alfabetizzazione civica che all'aritmetica.

3. **L'apprendimento ha valore oltre l'utilità economica.** Non insegniamo la matematica solo perché tutti diventino matematici: sviluppa astrazione, precisione e ragionamento disciplinato. L'ingegneria del software può essere preziosa allo stesso modo anche se l'IA scrive il codice. Insegna a costruire sistemi formali, a rendere esplicite le assunzioni, a ragionare sui fallimenti, a coordinarsi con altri e a trasformare intenzioni vaghe in strutture operative. Il valore educativo può sopravvivere anche quando cambia il valore di mercato del lavoro.

4. **Non sappiamo davvero cosa significhi «meglio».** «L'IA lo fa meglio» di solito significa meglio secondo una metrica scelta: velocità, costo, accuratezza su un benchmark, tasso di difetti, ricavi. Ma la qualità del software è plurale: manutenibilità, contestabilità, trasparenza, resilienza, autonomia locale, privacy, equità, reversibilità, comprensibilità umana. Un'IA potrebbe produrre meno bug creando un sistema meno governabile, o ridurre i costi aumentando la dipendenza da un unico fornitore. Lo stesso caso METR lo mostra: la velocità *percepita* e quella *misurata* possono divergere in modo marcato (Becker et al., 2025). «Meglio di noi» richiede sempre una domanda successiva: meglio secondo chi, per quale scopo, sotto quali vincoli, con quale distribuzione di benefici e danni?

5. Ma forse l'ossessione stessa è la risposta sbagliata

L'obiezione mette in luce un pericolo reale: gli educatori potrebbero difendere l'insegnamento dell'ingegneria del software soprattutto perché sono legati alla professione attuale, e sarebbe una ragione debole. Ecco cosa non vale la pena preservare:

- il codice scritto a mano come rituale fine a sé stesso;
- gli esercizi di sintassi tediosi;
- i compiti artificiali che ignorano gli strumenti reali;
- l'assunzione che ogni studente debba diventare un programmatore professionista;
- l'idea che l'occupabilità sia l'unica giustificazione dell'educazione.

L'insegnamento della programmazione dovrebbe probabilmente spostarsi dalla produzione verso l'interpretazione, la sperimentazione, la verifica e la comprensione dei sistemi. Può farlo su tre livelli, senza che ogni studente debba raggiungere l'ultimo:

Livello	Scopo educativo futuro
Usare il software	Comprendere e dirigere sistemi costruiti dall'IA
Costruire software	Progettare, testare e governare i sistemi
Studiare il calcolo	Comprendere cosa può essere formalizzato, automatizzato e ottimizzato

Non tutti hanno bisogno del secondo livello. Ma una società in cui nessuno lo raggiunge diventa vulnerabile a chiunque controlli i costruttori automatizzati.

6. La possibilità radicale: educare per la vita dopo il lavoro

Alla fine la domanda va oltre l'ingegneria del software. Se l'IA può svolgere gran parte del lavoro cognitivo economicamente rilevante, l'educazione non può restare organizzata principalmente attorno all'occupabilità. Le domande vere diventano altre:

- Come si danno le persone i propri scopi quando la produttività smette di essere la misura principale del valore?
- Come partecipano i cittadini alle decisioni prese da sistemi che non possono costruire personalmente?
- Come si distribuiscono status, reddito e contributo sociale quando non sono più legati al lavoro?
- Cosa resta prezioso quando non è economicamente necessario?
- Come mantengono le persone la propria capacità d'agire in un mondo saturo di assistenza intelligente?

In quel mondo, l'educazione all'ingegneria del software potrebbe sopravvivere non come formazione professionale ma come formazione civica e filosofica per una civiltà ingegnerizzata. Servirebbe abbastanza informatica perché i cittadini comprendano i sistemi che modellano le loro vite, così come le persone hanno bisogno di una certa comprensione del diritto o dell'economia senza diventare avvocati o economisti.

7. Una stella polare educativa migliore

«Insegnare ciò che l'IA non sa fare» è un principio troppo debole. Quello più forte è:

Insegnare ciò di cui gli esseri umani hanno bisogno per vivere in modo intelligente e libero con sistemi che potrebbero essere più capaci di loro.

Questo include la comprensione tecnica, ma anche:

- il giudizio sugli scopi;
- la consapevolezza dell'incertezza;
- la capacità di mettere in discussione l'autorità;
- la capacità di cooperare;
- l'immaginazione etica e politica;
- la tolleranza per l'ambiguità;
- la capacità di assumere e difendere impegni;
- la comprensione della dipendenza e del potere.

Proposte recenti vanno in una direzione simile. Ehlers, ad esempio, indica come dimensioni chiave dell'apprendimento nell'era dell'IA l'umiltà epistemica, il ragionamento etico, l'opposizione creativa, il pensiero relazionale e l'autonomia digitale (Ehlers, *J. of Innovative Business and Management*, 2026).

C'è uno strato ancora più radicale: alcune di queste capacità potrebbero un giorno essere esercitate meglio dall'IA. In quel caso la giustificazione dell'educazione smetterebbe di riguardare il vantaggio comparato e diventerebbe una questione di partecipazione. Insegniamo a suonare anche se le registrazioni sono tecnicamente superiori, a disegnare anche se i generatori di immagini sono più veloci, a fare matematica anche se le calcolatrici sono più affidabili nell'aritmetica. L'attività può contare perché cambia la persona che la svolge, non perché l'artefatto che ne risulta vince una gara contro una macchina.

La risposta a «cosa resta?» potrebbe quindi non essere una funzione umana speciale che l'IA non potrà mai riprodurre. Potrebbe essere semplicemente **l'esperienza umana di comprendere, scegliere, partecipare e assumersi la responsabilità**, anche quando una macchina potrebbe svolgere il compito corrispondente in modo più efficiente. È una risposta meno confortante di «l'IA non capirà mai davvero», ma più solida, se la premessa che l'IA continuerà a migliorare si rivelerà corretta.

Riferimenti (solo fonti verificate e citate nel testo)

Numerazione tra parentesi quadre = numero nell'elenco originale del PDF. † = fonte aggiunta in fase di verifica.

Linee guida e curricula - [1] ACM/IEEE-CS/AAAI CS2023 Task Force. *Generative AI and the Curriculum* (bozza, gen. 2024). <https://csed.acm.org/wp-content/uploads/2024/01/Generative-AI-v1.pdf> - [2] ACM/IEEE-CS/AAAI CS2023 Task Force. *Generative AI and the Curriculum*, §4.3 (apr. 2024). <https://csed.acm.org/wp-content/uploads/2024/04/4.3-Generative-AI-and-the-Curriculum.pdf> - [7] Kumar, A. N., Raj, R. K., et al. *Computer Science Curricula 2023*. ACM, 2024. doi:10.1145/3664191. <https://dl.acm.org/doi/pdf/10.1145/3664191> - [8] IEEE Computer Society. *New CS2023 Curriculum Guide* (comunicato stampa, 5 giugno 2024). <https://www.computer.org/press-room/new-cs2023-curriculum-guide> - [9] Kumar, A. N., Raj, R. K. *Computer Science Curricula 2023* (presentazione). <https://csed.acm.org/wp-content/uploads/2024/04/CS2023Presentation2.pdf>

Fonti storiche - [3] Knuth, D. E. *Literate Programming*. The Computer Journal 27(2):97–111, 1984. <https://www.cs.tufts.edu/~nr/cs257/archive/literate-programming/01-knuth-lp.pdf> (URL corretto: l'originale puntava a [cs.tufts.edu](https://www.cs.tufts.edu), dominio inesistente)

Studi empirici su produttività e apprendimento - [4] / [33] Peng, S., Kalliamvakou, E., Cihon, P., Demirer, M. *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot*. arXiv:2302.06590, 2023. <https://arxiv.org/abs/2302.06590> — riassunto: <https://github.com/AvneeshSarwate/ai-econ-research/blob/main/summaries/peng-2023-github-copilot-productivity.md> - [5] Becker, J., Rush, N., Barnes, E., Rein, D. (METR). *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. arXiv:2507.09089, 2025. <https://arxiv.org/abs/2507.09089> — resoconto: <https://scienceblog.com/t-a-randomized-trial-by-metr-found-that-experienced-developers-completed-real-coding-tasks-19-slower-when-allowed-to-use-ai-tools-yet-afterwards-they-estimated-on-average-that-ai-had-made-them-20-faster/> - † METR. *We are Changing our Developer Productivity Experiment Design*, 24 febbraio 2026. <https://metr.org/blog/2026-02-24-uplift-update/> - [6] Chen, R., Jiang, S., Shen, J., Moon, A., Wei, L. *Examining the Usage of Generative AI Models in Student Learning Activities for Software Programming*. arXiv:2511.13271, 2025. <https://arxiv.org/abs/2511.13271> - [13] Jang, E.-S., Oh, K.-S. *Investigation of the Influence of Flipped Learning and Text-Based Generative AI in Programming Education*. Computer Applications in Engineering Education, 2026. doi:10.1002/cae.70146 - [14] Hidalgo García, P., Mijač, M., Domínguez Díaz, A., Rodríguez García, D. *A preliminary study on the use of generative AI in software engineering education*. CECIIS 2025. <https://portalcientifico.uah.es/documentos/696a78519e41074a913019fd> - [15] Grewe, D., Vedislav, R.-M., Scherz, W. D. *Using Generative AI in Higher Programming Education: An Empirical Evaluation*. CSEDU 2026. <https://www.scitepress.org/PublishedPapers/2026/147267/> - [17] Mircea, M., Schmid, E., Droste, J., Schneider, K. *How Do Software Engineering Students Use Generative AI in Real-World Capstone Projects?* arXiv:2604.24521, 2026. <https://arxiv.org/html/2604.24521v1> - [18] Qin, Q., de Souza Santos, R., Spinola, R. *On the Role and Impact of GenAI Tools in Software Engineering Education*. arXiv:2512.04256, 2025. <https://arxiv.org/html/2512.04256v1> - [19] Melgarejo-Solis, R., et al. *Artificial Intelligence (ChatGPT) and Its Impact on the Academic Success of Software Engineering Students at the UNMSM*. LACCEI/LEIRD 2025. <https://proceedings.laccei.org/index.php/laccei/article/view/4885> - [20] Yalwa, A. S., Othman, M. S., Yusuf, L. M., Almarshadi, M. S. *Generative AI in Higher Education: A*

Systematic Review with Emphasis on Programming and Computer Science Education. IJACSA 17(5), 2026. <https://thesai.org/Publications/ViewPaper?Volume=17&Issue=5&Code=IJACSA&SerialNo=31> - [21] Huang, T.-C., Tseng, H.-P. *Learning, Behavior, and Pedagogy: A Systematic Review of Generative AI Use in Programming Education*. IJIE 16(1):102–116, 2026. (Link originale: post LinkedIn dell'autore) https://www.linkedin.com/posts/tien-chi-huang-4820a5237_international-journal-of-information-and-activity-7418588849066545152-9kvl - [22] Beralde, P. N. T., et al. *Utilization of Artificial Intelligence and Perceived Programming Skills of Third Year IT Students at Quezon City University*. IJRSI 13(5), 2026. <https://rsisinternational.org/journals/ijrsi/view/utilization-of-artificial-intelligence-and-perceived-programming-skills-of-third-year-information-technology-students-at-quezon-city-university>

Didattica e curricula di ingegneria del software con IA - [10] Azemi, A. *Teaching Programming in the Age of AI: Transforming Pedagogy Amidst Code-Generating Technologies*. IEEE FIE 2025. doi:10.1109/FIE63693.2025.11328573. <https://www.computer.org/csdl/proceedings-article/fie/2025/11328573/2df9tfS6Tte> - [12] Wang, T. *Exploring Generative AI for Pedagogically Aligned Learning Experiences and Adapted Instructional Practices in Software Engineering Education*. Tesi di dottorato, Virginia Tech, 2026. <https://vtechworks.lib.vt.edu/items/10baf083-8c27-4c72-8305-7091721b42fe> - [16] Bouvier, D. J., et al. *The Rest of the Robots: Generative AI in Post-introductory Computing Education*. ITiCSE-WGR 2025. <https://researchportal.northumbria.ac.uk/en/publications/the-rest-of-the-robots-generative-ai-in-post-introductory-computi/> - [23] Garousi, V., et al. *Encouraging responsible GenAI use in software engineering education: A design-oriented model*. Journal of Systems and Software, 2026 (preprint: arXiv:2506.00682). https://www.academia.edu/165180707/Encouraging_responsible_GenAI_use_in_software_engineering_education_A_design_oriented_model - [24] Randall, N., Wäckerle, D., Stein, N., Goßler, D., Bente, S. *What an AI-Embracing Software Engineering Curriculum Should Look Like: An Empirical Study*. IEEE Software 41(2):36–43, 2024. doi:10.1109/MS.2023.3344682 - [25] Geng, F., Shah, A., Chen, M., Denny, P., Leinonen, J., Griswold, B., Soosai Raj, G., Porter, L. *Mapping the Emerging Curriculum for AI-Assisted Software Engineering via Syllabus Analysis*. arXiv:2608.05898, 2026. <https://arxiv.org/pdf/2608.05898v1.pdf>

Filosofia dell'educazione, società, economia - [46] UNESCO. *AI and the future of education: Disruptions, dilemmas and directions*. 2025. <https://www.unesco.org/en/articles/ai-and-future-education-disruptions-dilemmas-and-directions> - [47] Hinton, G., intervista a *StarTalk* (trascrizione), 2026. <https://singjupost.com/is-ai-hiding-its-full-power-w-geoffrey-hinton-transcript/> - [49] Bialystok, L. *AI and the Future of (Philosophy of) Education*. Educational Theory 76:133–139 (online 5 ottobre 2025). doi:10.1111/edth.70058 — record: <https://philpapers.org/rec/BIAAAT-3> (link originale: <https://www.scribd.com/document/1018176014/Educational-Theory-2025-Bialystok-AI-and-the-Future-of-Philosophy-of-Education>) - [52] Ehlers, U.-D. *How Artificial Intelligence is shaping the future of learning: rethinking education, competence, and human agency*. Journal of Innovative Business and Management 18(1), 2026. <https://journal.doba.si/jimb/en/article/view/427> - [56] Acemoglu, D., Autor, D., Johnson, S. *Building Pro-Worker Artificial Intelligence*. NBER Working Paper 34854, 2026. <https://www.nber.org/papers/w34854>

Fonti filosofiche primarie e di riferimento (†, aggiunte in verifica) - † Platone, *Fedro* 275a–e, trad. H. N. Fowler (1925). Perseus Digital Library. <https://www.perseus.tufts.edu/hopper/text?doc=Perseus:text:1999.01.0174:text=Phaedrus:page=275> - † Aristotele, *Etica Nicomachea* VI, 1140a, trad. H. Rackham. Perseus Digital Library. <https://www.perseus.tufts.edu/hopper/text?doc=Perseus:text:1999.01.0054:bekker%20page=1140a> - † Arendt, H. *Thinking and Moral Considerations: A Lecture*. Social Research 38(3), 1971. <https://jonudell.net/h/arendt.pdf> - † Stanford Encyclopedia of Philosophy: *Plato's Ethics* (<https://plato.stanford.edu/entries/plato-ethics/>), *Aristotle's Ethics* (<https://plato.stanford.edu/entries/aristotle-ethics/>), *Martin Heidegger* (<https://plato.stanford.edu/entries/heidegger/>), *Hannah Arendt* (<https://plato.stanford.edu/entries/arendt/>)

plato.stanford.edu/entries/arendt/) - † Wikipedia: *Tools for Conviviality* (https://en.wikipedia.org/wiki/Tools_for_Conviviality), *Bernard Stiegler* (https://en.wikipedia.org/wiki/Bernard_Stiegler)